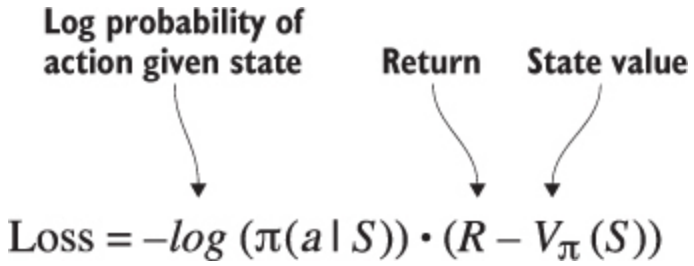
These problems are related, since the reward variance depends on how many samples we collect (more samples yields less variance). The idea behind a combined value-policy algorithm is to use the value learner to reduce the variance in the rewards that are used to train the policy. That is, instead of minimizing the REINFORCE loss that included direct reference to the observed return, *R*, from an episode, we instead add a baseline value such that the loss is now:



Here, $V(S)$ is the value of state S, which is the state-value function (a function of the state) rather than an action-value function (a function of both state and action), although an action-value function could be used as well. This quantity, $V(S) - R$, is termed the *advantage*. Intuitively, the advantage quantity tells you how much better an action is, relative to what you expected.

Note

Remember that the value function (state value or action value) implicitly depends on the choice of policy, so we ought to write $V_\pi(S)$ to make it explicit; however, we drop the π subscript for notational simplicity. The policy's influence on the value is crucial, since a policy of taking random actions all the time would result in all states being of more or less equally low value.

Imagine that we're training a policy on a Gridworld game with discrete actions and a small discrete state-space, such that we can use a vector where each position in the vector represents a distinct state, and the element is the average rewards observed after that state is visited. This look-up table would be the $V(S)$. We might sample action 1 from the policy and observe